Swiggy Squad

Order together. Split as you go.
Settle without the spreadsheet.

A proposal for the Swiggy Builders Club

Lives inside: Instamart · Food · Subscriptions

What it is: A feature layer, not a separate app

Swiggy Squad

Order together. Split as you go. Settle without the spreadsheet.

A proposal for a collaborative ordering and expense-sharing layer inside Swiggy.

Submitted for	Swiggy Builders Club
Lives inside	Instamart · Food · Subscriptions
What it is	A feature layer, not a separate app
Status	Concept + architecture, ready to build

Contents

1. Where this came from
2. The problem
3. Why now
4. What Swiggy Squad is
5. How it behaves
6. Walking through it
7. What ships first
8. How it's built
9. Data model
10. APIs & MCP integration
11. The real-time cart
12. The money engine
13. Scaling it
14. The hard cases
15. What's in it for Swiggy
16. Making money
17. Why people stay
18. What could go wrong
19. The honest case for building it
20. Roadmap

1. Where this came from

I live in a shared flat. Every week the same thing happens: someone orders milk and eggs on Instamart, someone else throws in snacks, a third person needed rice. One person's card gets charged for all of it. Then comes the annoying part — a WhatsApp thread to figure out who owes what, a Splitwise entry someone forgets to make, and a week later nobody quite remembers whether the ₹340 was settled or not.

The ordering is the smooth part. Everything around the money is broken, and it's broken in a very specific way: every tool that splits expenses works *after* the purchase is done. Splitwise, the group chat, the UPI request. All of it is cleanup, after the fact, in a different app.

Swiggy is the one place that sees the order *as it's happening*. That's the whole idea behind Squad:

> *Split the bill during the order, not after it. No re-entry, no second app, no forgotten ₹340.*

Because Swiggy already owns the catalog, the payment rails, and the delivery, it's the only player that can actually do this. An expense-tracking app can't suddenly become a grocery platform. A grocery platform that hasn't built this yet is leaving a real retention moat sitting on the table.

The rest of this document lays out what the feature is, how it works, how I'd build it on top of Swiggy's existing infrastructure, and the honest case for why it's worth Swiggy's time (and where it's risky).

2. The problem

Group ordering isn't something Swiggy needs to convince anyone to do. It's already happening, everywhere, every day. Flatmates split a grocery run. An office floor pools a lunch order and one person pays for fourteen meals. Friends on a trip rack up a shared tab nobody's tracking. The ordering works fine. It's the money around it that falls apart.

Walk through what a normal shared-flat grocery run actually looks like today:

Step	Where it happens	What goes wrong
Decide what to buy	WhatsApp group	Messages scroll past; someone's request gets missed
Place the order	Swiggy Instamart	One person's card eats the whole bill
Note who owes what	Splitwise, or memory	Manual entry; usually skipped
Work out the split	Mental math	"Wait, who had the protein powder?"
Settle up	GPay / PhonePe	Chasing people; the awkward reminder

Five steps, four apps, and at least three places where money quietly leaks out of the system — a forgotten item, a skipped Splitwise entry, a balance nobody remembers. In a flat where this repeats every week, it's a low-grade but constant source of friction. People genuinely fall out over ₹200.

The tools that exist all solve one slice and miss the rest. Splitwise is a ledger you fill in after the fact — it never sees the actual order and can't tell who bought what. UPI moves the money but has no idea what it's for. The group chat coordinates but leaves no record. None of them sit at the one moment that matters: the checkout, where the order and the money are the same event. That spot is empty, and it's the only spot from which this problem is actually solvable.

---

3. Why now

A feature like this could have been imagined five years ago, but it wouldn't have worked. Four things have lined up that make this the right moment — and they happen to line up exactly on Swiggy's core user.

Start with the obvious: food delivery in India is huge and mature. The market was worth around **$55.6 billion in 2025**, and it's settled into a stable two-horse race — Zomato at roughly 57% of order value, Swiggy at 43%, a gap that's barely moved through FY25 and into FY26. When the market stops being won by discounting, it gets won by getting more out of the users you already have. That's the lever Squad pulls.

Quick commerce is where it really bites, though, because groceries are *planned and repeated* in a way a Friday biryani isn't. Instamart's space has exploded — India's q-commerce GMV hit about **$7.6 billion in FY2025, more than double the year before**, and quick commerce now accounts for roughly **two-thirds of all online grocery orders** in the country. Instamart itself runs **1,100+ dark stores across 100+ cities** at ~13-minute delivery. But it's in a knife-fight for share: Blinkit's out front near 48%, with Instamart around 24% and Zepto at 22%. In a race that tight, anything that makes users *stickier* is worth more than its direct revenue — and the heaviest q-commerce users are 18-24, which is precisely who lives four-to-a-flat.

Which brings up the third thing. Squad's user lives with other people, and that population is swelling. Co-living bed demand in India was tracking toward **6.6 million in 2025** and an estimated 9.1 million by 2030. Around **50 million migrants aged 20-34** have moved to cities to study or work; higher-ed enrolment sits near 43 million students against housing for maybe 4 million, which dumps the rest into PGs, hostels, and shared flats — concentrated in Bengaluru, Hyderabad, Pune, Mumbai, Delhi-NCR, Chennai. Swiggy's strongest cities, in other words.

And finally, the part Swiggy doesn't have to build at all: settlement. UPI did **228 billion transactions worth nearly ₹300 lakh crore in 2025** — about 698 million a day — and the average ticket has dropped to ~₹1,293 as people use it for everyday small stuff. "I'll UPI you later" is now a complete sentence everyone understands. Tellingly, industry trackers have noted bill-split features on UPI eating into Splitwise's relevance since 2025 — the behaviour Squad formalizes is *already* migrating onto rails Swiggy can plug into. Squad doesn't ask anyone to pay differently. It just makes the tracking smart and the settling painless.

Figures above are from public 2025-26 market research and reporting; sources are listed at the end.

---

4. What Swiggy Squad is

Squad is a layer that lives inside the Swiggy app — not a new app, not a new login, not a tab solo users ever have to see. It only shows up once you're part of a squad, and stays out of the way otherwise, so the person who just wants to order one biryani at 11pm never feels any of it.

A *squad* is a standing group of people who order together — your flat, your office floor, a trip group. Each one keeps its own shared cart, its own ledger, its own subscriptions, its own settlement history, all walled off from every other squad you're in. (You're in several at once; more on that below.)

Three things hold it together. First, **a shared cart everyone builds at the same time** — Raj adds milk, Aanya adds eggs, you add rice, and everyone sees who added what as it happens. Second, **the split is decided right there at checkout** — equal, custom, or down to the individual item — so there's nothing to re-enter into a second app afterwards. Third, **a running ledger that settles on everyone's own time** — UPI or confirmed cash, no one forced to square up the instant an order lands.

What makes this Swiggy's to build and nobody else's is that the hard parts already exist here:

Squad needs	Swiggy already has it
A product catalog	Food + Instamart
A live cart	Existing cart service, extended
Payments	Swiggy's payment layer + UPI
Delivery tracking	Already built
Recurring orders	Instamart subscriptions

Squad bolts onto the spine Swiggy already operates. Anyone trying to build this from outside would have to recreate all five of those first — which is exactly why an expense app can't, and won't.

---

5. How it behaves

The Cart Pilot

The trickiest design question in any group-ordering feature is *who's in charge of an order*. The wrong answer is a single permanent admin who has to approve everything — that turns one person into a bottleneck and dumps all the nagging-everyone work on them. So Squad doesn't do that.

Instead, **whoever starts an order is the Cart Pilot for that order, and only that order.** The role is per-order and it rotates naturally — whoever's doing tonight's run is in charge of tonight's run. The Pilot owns their cart end to end: they confirm the items, decide who's splitting what, set the split, place the order, fix the split afterwards if someone got tagged wrong, and confirm any cash that comes back. Nobody can reach into their order and change it.

Separately, there's a **Squad Admin**, but the Admin only handles the *group* — inviting people, removing people, handing over the role when they leave. Crucially, the Admin has **no power over any order they didn't pilot themselves.** They can't edit it, can't cancel it, can't override the split. This is deliberate: the person who placed an order is the only one who should be able to touch it.

Role	Covers	Can do
Cart Pilot	One cart / order	Everything about that order — items, split, checkout, post-order fixes, cash confirmation. Exclusive; nobody overrides it.
Squad Admin	The squad	Invites, removals, roles, succession, reminders, settings. Zero authority over orders they didn't pilot.
Member	Default	Add to a shared cart, mark own items personal, fork an idle cart, see the ledger, settle up.

What happens to a cart nobody finishes

Because a cart belongs entirely to its Pilot, no one can wrestle it away — but people still need to get their groceries when the Pilot goes quiet. The answer is forking, not seizing.

A Pilot only keeps one open cart at a time; to start over they edit it or bin it, so stale carts don't pile up. If they open a cart and wander off without ordering, it just sits there under their name — safe, untouched. Now suppose Raj needs to order *now* and Priya's half-built cart is still sitting open. He doesn't take over her cart. He **forks** it — copies everything into a fresh cart that's his — and pilots that one himself. The copied items keep their original "added by" tags, so the split still knows the milk was Priya's idea. If Priya later comes back and checks out her original, Squad warns her that a near-identical cart already went out ("a similar order was already placed by Raj — go ahead anyway?") but doesn't stop her. And the moment a fork happens, a small note drops in the squad chat so nobody's confused about which cart is the live one.

Squad chat

Every squad has a chat thread, because the "anyone want anything? I'm ordering" conversation has to happen *somewhere* — and right now it happens on WhatsApp, away from the cart. Pulling it inside Swiggy means the discussion sits right next to the thing it's about.

Being in more than one squad

Nobody belongs to just one group. You've got the flat, the office lunch crowd, the friends you travel with. Squad treats each as fully separate — its own cart, ledger, subscriptions, history — and switching between them is a single tap. The split logic for your office lunch has nothing to do with your flat's grocery ledger, and it shouldn't.

---

6. Walking through it

The best way to see whether this hangs together is to follow one real order start to finish.

Setting it up. Priya creates "Home Squad" from the Squad tab — names it, picks the type, pulls in Raj and Aanya from her Swiggy contacts or just shares an invite link. Whole thing takes under a minute. That invite link is quietly doing double duty: if Raj weren't on Swiggy yet, it'd land him on an install page pitched around something he already does with his flat, which converts a lot better than a generic "₹100 off your first order" referral.

The weekly run. Priya opens Home Squad and starts an Instamart order — that makes her the Cart Pilot. She drops a line in the squad chat: *"doing the grocery run, add what you want."* Raj and Aanya open the same cart and start dropping things in. Every item shows who added it, so there's no mystery about whose protein powder that is. When Raj adds milk that Priya already put in, he gets a quick prompt — combine into one shared item, or keep two? He can also flag his protein powder as "just mine." When everyone's done, Priya hits the split screen: shared stuff defaults to an even split, and she nudges the item-level bits as needed (Raj's protein powder is all his, the snacks are just her and Aanya). She pays, and the ledger updates the instant the order's placed. All three of them track the one delivery.

And if Priya gets pulled into a meeting and never checks out? Her cart just waits. Raj, who needs eggs now, forks it — everyone's items copy over with their names intact — and he pilots his own order. The chat gets a note so nobody double-buys.

Settling up. Nobody has to pay the second an order lands — balances just accrue in the ledger. When Raj wants to clear his, he settles over UPI (the squad keeps everyone's UPI IDs) or hands Priya cash and she confirms she got it, which closes the balance. Open balances get a gentle monthly nudge; if someone's really dragging, the Admin can send *one* private reminder (never a group call-out), and there's a cooldown so it can't become nagging.

The recurring stuff. Somebody sets up the daily milk and the weekly water cans once — daily, weekly, whatever cadence — and the cost auto-shares across whoever's on it. If the person who set up the milk moves out, the subscription doesn't vanish; it hands off to someone still in the flat and the cost re-shares. Nothing silently breaks.

---

7. What ships first

I'd resist the urge to build all of this at once. The thing that makes Squad worth anything — the part that earns the "oh, that's clever" reaction — is splitting the bill *inside* the order. Everything else is supporting cast. So phase one is ruthless about shipping that one loop well and skipping anything that drags in payments regulation or operational weight.

Phase 1 — the core that has to be undeniable

What	Why it's non-negotiable for v1
Squad creation + invites	Nothing works without it
Live shared cart with attribution	This is the visible magic
Squad chat	The "anyone want anything?" has to live here
Cart Pilot split at checkout	*The* idea — splitting during the order
Equal / custom / item-level split	Where the actual pain dies
A basic, read-only ledger	People won't trust a split they can't see
Manual settlement (UPI / cash confirm)	Keeps pooled money — and RBI — out of v1

Phase 2 — make them stay: recurring subscriptions with shared cost, the squad savings counter (the "you saved ₹430 this month" surface, which I'd prioritise since it's cheap to build and the strongest retention hook), price-drop nudges, settlement reminders, monthly reports with a free-tier export cap, and the admin-succession logic.

Phase 3 — make it smart: split suggestions learned from a squad's history, market-rate comparisons layered onto the savings view, idle-squad re-engagement vouchers tied to the health signal, squad demand signals feeding Instamart's forecasting, and proper billing for office squads.

The reason for holding pooled money until later isn't timidity — it's that a squad wallet touches RBI's prepaid-instrument rules, and there's no reason to take that on before the core loop has proven people even want this. A ledger plus out-of-band UPI does the job for v1 without the regulatory overhead.

---

8. How it's built

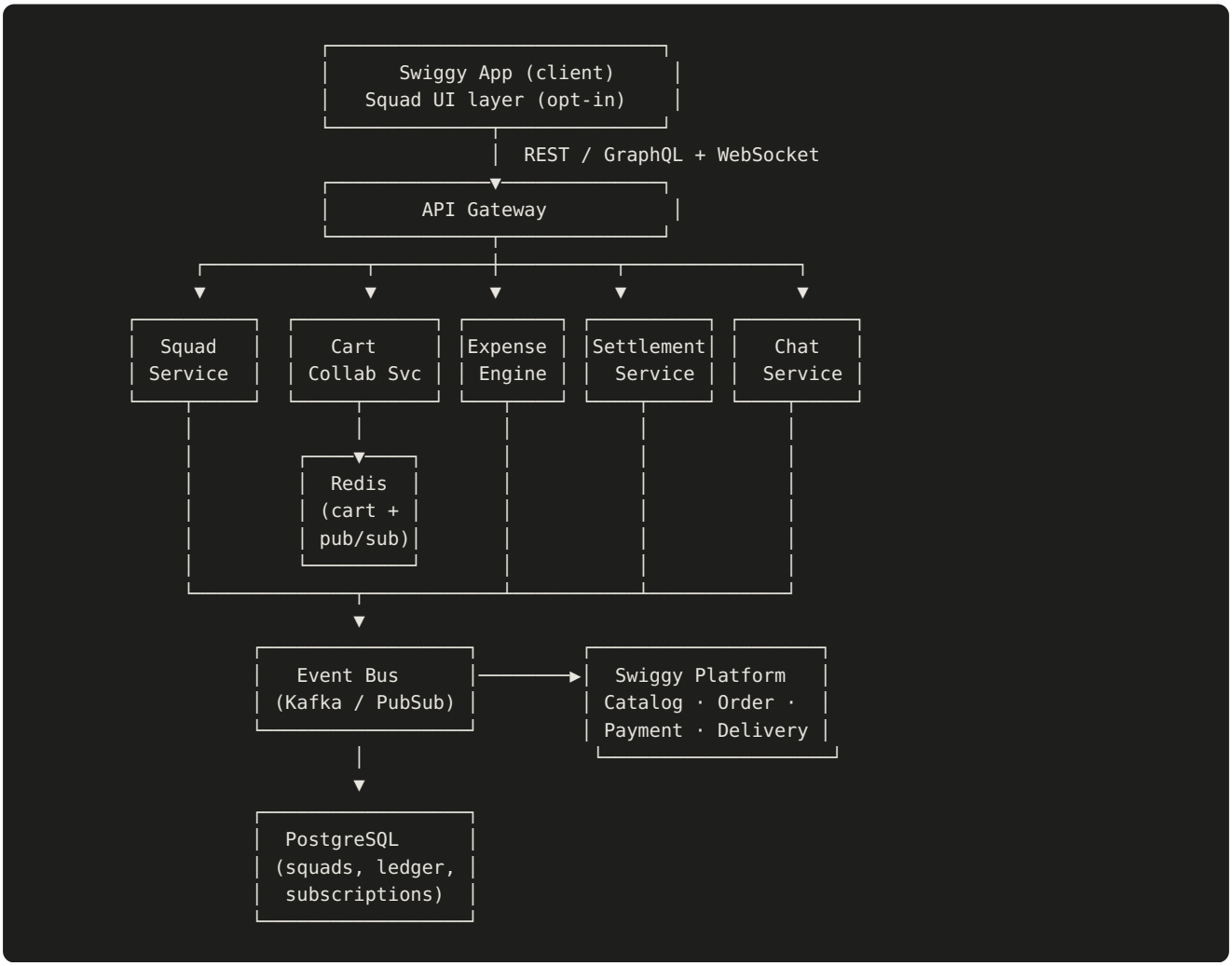
The guiding constraint is that Squad shouldn't fork or fight Swiggy's existing platform — it should sit on top of it. So it's a handful of focused services, each owning its own data, talking to each other through normal APIs for commands and an event bus for everything that needs to ripple across services without tight coupling.

The services

Service	What it does	Owns
Squad	Squad lifecycle, membership, roles, succession	Squads, members, roles
Cart Collaboration	Live shared-cart state, attribution, conflicts	Carts, cart items
Expense Engine	Computes splits, writes ledger entries, balances	Ledger, splits, balances
Settlement	UPI/cash settlement, reminders, closing balances	Settlements, reminders
Subscription	Recurring essentials, cost-sharing, ownership handoff	Subscriptions
Chat	Per-squad messaging	Messages
Notification	Push, reminders, squad events	(stateless)

These don't replace anything — they call into Swiggy's existing Catalog, Order, Payment, and Delivery Tracking services.

The shape of it



Why the event bus matters

The cart, the ledger, and the order all have to stay in sync without being welded together. The clean way to do that is events. When an order goes through, the Order Service fires an `order.placed` event; the Expense Engine picks it up and freezes the split into ledger rows, and the Subscription Service picks up the same event to roll its schedules forward. Nothing blocks on anything else — if the ledger write lags by a second, the order still completes and the ledger catches up. That decoupling is what keeps a money feature from becoming a single point of failure on the ordering path.

---

9. Data model

The core tables, roughly. A few are worth calling out — the rest are what you'd expect.

```
Squad
  ├─ id (uuid, PK)
  ├─ name
  ├─ type (home | office | trip | custom)
  ├─ admin_id (FK → Member)
  ├─ created_at
  └─ status (active | archived)

Member
  ├─ id (uuid, PK)
  ├─ swiggy_user_id (FK → Swiggy user)
  ├─ display_name
  └─ upi_id (nullable, self-entered per squad)

SquadMembership   (join: which user is in which squad + their role)
  ├─ squad_id (FK)
  ├─ member_id (FK)
  ├─ role (admin | member)
  ├─ balance (computed; cached)
  └─ joined_at

SharedCart
  ├─ id (uuid, PK)
  ├─ squad_id (FK)
  ├─ pilot_id (FK → Member, the initiator)
  ├─ forked_from (nullable FK → SharedCart)   # set when copied from an idle cart
  ├─ status (building | locked | ordered | discarded)
  └─ updated_at
  # Constraint: at most ONE cart in `building` status per (squad_id, pilot_id).
  # Multiple concurrent carts may exist in a squad, each owned by a different Pilot.

CartItem
  ├─ id (uuid, PK)
  ├─ cart_id (FK)
  ├─ catalog_item_id (FK → Swiggy catalog)
  ├─ added_by (FK → Member)
  ├─ quantity
  ├─ is_personal (bool)         # "100% mine"
  └─ split_participants (FK list → Member)

Order
  ├─ id (uuid, PK)
  ├─ squad_id (FK)
  ├─ cart_id (FK)
  ├─ pilot_id (FK → Member)
  ├─ swiggy_order_id (FK → Swiggy order)
  ├─ paid_by (FK → Member)
  ├─ total_amount
  └─ split_rule_snapshot (jsonb)   # immutable copy at checkout

LedgerEntry
  ├─ id (uuid, PK)
  ├─ squad_id (FK)
  ├─ order_id (FK)
  ├─ member_id (FK)
  ├─ amount_owed
  ├─ amount_paid_by_member
  └─ created_at

Settlement
  ├─ id (uuid, PK)
  ├─ squad_id (FK)
  ├─ from_member (FK)
  ├─ to_member (FK)
```

```
  ├─ amount
  ├─ method (upi | cash)
  ├─ status (pending | partial | complete)
  └─ confirmed_by (FK → Member)    # payer/Pilot confirms cash

Subscription
  ├─ id (uuid, PK)
  ├─ squad_id (FK)
  ├─ owner_id (FK → Member)
  ├─ catalog_item_id (FK)
  ├─ cadence (daily | weekly | custom)
  ├─ shared_among (FK list → Member)
  └─ status (active | paused | cancelled)
```

The one entity that needs care: the split rule. It's freely editable while a cart is being built, then frozen the moment the Pilot checks out — serialized into `Order.split_rule_snapshot` and written out as `LedgerEntry` rows. After that it's treated as locked, with exactly one exception: the Pilot can correct an honest mistake (wrong person tagged) afterwards, and when they do, the change is logged and everyone affected gets notified. The point is that the ledger can be corrected but never quietly rewritten — if your balance moves, you find out why.

How it all relates:

```
Squad 1──∞ SquadMembership ∞──1 Member
Squad 1──∞ SharedCart 1──∞ CartItem
Squad 1──∞ Order 1──∞ LedgerEntry
Squad 1──∞ Settlement
Squad 1──∞ Subscription
Order ∞──1 Member (pilot / paid_by)
```

---

10. APIs & MCP integration

This is the part that ties directly to the Builders Club: if Squad gets access, it would talk to Swiggy's MCP servers (Food and Instamart) for catalog, cart, and order operations, and layer its own services — collaboration, expense, settlement — on top. Squad never reaches into Swiggy's internal tables; it goes through the published interface and reacts to events. A representative slice of the API:

```
# Squad lifecycle
POST   /v1/squads                      Create a squad
POST   /v1/squads/{id}/invites         Invite a member
POST   /v1/squads/{id}/members/{m}/role  Change role / designate successor
DELETE /v1/squads/{id}/members/{m}     Remove member (blocked if balance open)

# Collaborative cart
POST   /v1/squads/{id}/cart/items      Add item (returns dup-detection prompt if needed)
PATCH  /v1/squads/{id}/cart/items/{i}  Update qty / mark personal / set participants
DELETE /v1/squads/{id}/cart/items/{i}  Remove item (Pilot or item owner)
POST   /v1/squads/{id}/cart/lock       Pilot locks cart to begin split
POST   /v1/squads/{id}/cart/{c}/fork   Fork an idle cart → new cart, caller becomes Pilot
DELETE /v1/squads/{id}/cart/{c}        Discard own unordered cart (Pilot only)

# Split & order
POST   /v1/squads/{id}/cart/split      Set split rule (equal | custom | item-level)
POST   /v1/squads/{id}/orders          Place order (Pilot) → calls Swiggy Order via MCP

# Ledger & settlement
GET    /v1/squads/{id}/ledger          Balances + history (filterable by month)
POST   /v1/squads/{id}/settlements     Initiate UPI / record cash settlement
POST   /v1/settlements/{s}/confirm     Payer/Pilot confirms cash receipt
POST   /v1/squads/{id}/reminders       Admin-triggered individual reminder (cooldown-limited)

# Subscriptions
POST   /v1/squads/{id}/subscriptions   Create recurring essential
PATCH  /v1/subscriptions/{s}           Pause / resume / transfer ownership
```

10.2 Example: placing an order via Swiggy MCP

```
// Squad's Order Service composes the shared cart into a single
// Swiggy order through the Instamart MCP server.
{
  "tool": "instamart.create_order",
  "input": {
    "items": [
      { "catalog_item_id": "amul_milk_1l", "qty": 2 },
      { "catalog_item_id": "eggs_30", "qty": 1 }
    ],
    "delivery_address_id": "addr_home_squad",
    "payment_method": "upi"
  }
}
// On success → emit order.placed event →
// Expense Engine writes immutable LedgerEntry rows from the split snapshot.
```

---

11. The real-time cart

The shared cart is both the hardest thing to build and the thing people will actually point at when they describe Squad to a friend. It has to feel like Google Docs — you add milk, your flatmate sees it appear a fraction of a second later. Get this laggy or flaky and the whole feature feels broken, no matter how good the split logic is.

The setup is a well-worn pattern, which is the point — nothing exotic to fail. Live cart state sits in **Redis**, keyed per cart, because it's fast and its pub/sub fans changes out naturally. Everyone looking at a cart

holds a **WebSocket**; a change writes to Redis first (so it's instantly consistent for everyone connected) and then drifts down to Postgres for durability. If the database write is a beat behind, nobody notices.

Two people adding at once is the normal case, not an error. There's no locking. Different items just show up, each tagged with who added them. The same item added twice is the only interesting case, and it gets a prompt to the second person — *"Amul Milk 1L is already in the cart (Priya added it). One shared, or keep two?"* — which is the real question: are we sharing one or did you each want your own? Matching is on the actual SKU, so "Milk 500ml" and "Milk 1L" never get confused for each other.

There's a subtle race lurking here — two people hitting checkout at the same time — but the Pilot model dissolves it. Since a cart belongs only to its Pilot, nobody can co-checkout or grab someone else's cart. If you want to order while a cart sits idle, you fork it into your own and check that out independently. No distributed lock, no "someone else is editing" dead-end, and the only safeguard needed is the double-order warning if the original Pilot later checks out a cart whose fork already went through.

On consistency, the split between the two halves of the system is intentional: cart edits are last-write-wins on quantity (attribution always preserved, never silently merged), while ledger writes are strict and transactional off the frozen split snapshot. Balances are just cached sums recomputed from the ledger. The live surface optimizes for *feel*; the money surface optimizes for *correctness*. Keeping those apart is the whole trick.

---

12. The money engine

When the Pilot checks out, the engine turns the cart into who-owes-what. Equal split is just the total over however many people. Custom lets the Pilot punch in amounts directly. The interesting one is item-level: each item carries its own list of who's in on it, and the engine adds up each person's share across everything they're part of, then folds in their slice of the shared fees like delivery. Concretely:

Item	Cost	Who's in	Each pays
Milk	₹60	Priya, Raj, Aanya	₹20
Protein powder	₹900	Raj only	₹900 (Raj)
Snacks	₹120	Priya, Aanya	₹60
Delivery	₹30	everyone	₹10
Total ₹1,110			**Priya ₹90 · Raj ₹930 · Aanya ₹90**

Every order drops one ledger row per person, tracking what they owe and what they've paid. The squad's balances are just those rows summed up, and you can slice the ledger by month, by person, by what's still outstanding, and by how much the squad saved on offers.

The deliberate choice on settlement is that **Squad never holds anyone's money.** No wallet, no pool — that's what keeps RBI's prepaid-instrument rules out of the picture for now. Members drop their UPI ID into

the squad once, and settling fires a request to the right person. Cash works too, which most apps pretend doesn't exist: you mark "paid cash," and the person who's owed confirms they got it, which closes the loop instead of leaving a phantom balance. And none of it is forced — balances run, partial payments are fine, you reconcile when you reconcile. The only nudges are a quiet monthly reminder on open balances and, if someone's really dragging it out, a single private reminder the Admin can send (never a group call-out, and rate-limited so it can't turn into pestering).

---

13. Scaling it

Squad rides on Swiggy's scale, so most of the hard infra problems are already solved upstream. The things specific to Squad that I'd watch:

Concern	How it's handled
Real-time fan-out	Redis pub/sub + WebSocket; cart in memory, persisted async
Cart write contention	Per-cart atomic ops in Redis; no global locks
Double-checkout race	Forking instead of shared checkout — sidesteps it entirely
Ledger correctness	Transactional writes off the frozen split snapshot
Balance reads at scale	Cached sums, recomputed when the ledger changes
Subscription scheduling	Event-driven, idempotent on `order.placed`
Oversized squads	Free tier capped at a sane member count; paginated history
Cross-service consistency	Event bus, at-least-once delivery, idempotent consumers

If there's one line to take away, it's this: the cart is allowed to be fast and eventually-persisted, the ledger is never allowed to be anything but exactly right. They're built differently on purpose, and keeping them apart is what lets Squad feel instant without ever fumbling someone's money.

---

14. The hard cases

A group-money feature lives or dies on trust, and trust is mostly about handling the awkward moments well — the flatmate who never pays, the person who rage-quits the squad, the milk subscription nobody owns anymore. Here's how each gets handled, and the thread running through all of them is the same: nudge, don't shame; be transparent; and never move money the group didn't agree to move.

Situation	What happens
Someone never settles	Private reminders + balances visible within the group as quiet accountability — never a public call-out
Someone leaves owing money	They can't leave until it's cleared — settle-before-exit
Split dispute after the order	Pilot can fix genuine errors; the change is logged and everyone affected is told
The Admin leaves	The squad doesn't die — they hand the role over, or it passes to an active member who accepts it
Subscription owner leaves	It transfers to someone still in the squad and the cost re-shares; never silently dropped
Same item added twice	A prompt: one shared, or two separate?
Wrong UPI ID	Stored IDs are shown plainly before you send, so you can eyeball the recipient
Paid in cash	The person owed confirms receipt to close it
Personal item tagged wrong	Owner re-marks before checkout; Pilot can correct after, with a notification
Pilot opens a cart, never orders	It just waits under their name — nobody can override it; they edit or bin it to start over
Someone else needs that idle cart	They fork it into their own and pilot it; items keep their original names; chat gets a note
Admin tries to kill another's order	Not allowed — the Admin has no power over orders they didn't pilot

15. What's in it for Swiggy

The user benefit is easy to see. The reason Swiggy should care is that a few of these benefits are genuinely strategic, not just pleasant — so it's worth walking through them properly rather than listing them.

Lock-in that money can't buy. A solo user can defect to Blinkit in the time it takes to open the app; nothing holds them. Someone four-deep in a squad is a different story. They've got a shared ledger with running balances, a couple of standing subscriptions, months of settlement history, and three other people relying on the same setup. Leaving means unwinding a small social and financial arrangement, not just switching an app. In a market where Instamart (~24% share) is fighting Blinkit (~48%) and Zepto (~22%), that kind of stickiness is worth more than a discount war, precisely because a competitor can't match it by spending more — there's no coupon that buys back a group habit.

Bigger, more frequent, more predictable orders. Group carts are larger than solo carts almost by definition — five people's groceries in one basket. And the recurring subscriptions are the real prize: they

convert one-off impulse buys into scheduled, forecastable revenue, which is exactly the shift quick commerce needs to actually turn a profit. On top of that, collapsing a flat's three separate Tuesday orders into one means fewer delivery trips and tighter routing, landing straight on Swiggy's largest cost line.

This is where the inventory and forecasting story gets interesting — and it deserves more than a sentence. Quick commerce lives and dies on the dark store. Stock too little of something and you stock out and lose the order; stock too much of a perishable and you eat the waste. The whole game is predicting tomorrow's demand for a two-kilometre radius, and today that prediction is mostly inferred from messy historical averages.

Squad changes the input. A squad with a standing daily-milk-and-eggs subscription isn't a *prediction* — it's a near-certainty. The dark store serving that pincode knows, before the day starts, that it will move thirty litres of milk and a few dozen eggs to these specific addresses tomorrow morning. Multiply that across thousands of squads in a city and a meaningful slice of demand stops being a forecast and becomes a standing order book. That lets Swiggy:

- pre-position the right stock in the right dark store the night before, instead of guessing;
- cut both stockouts (lost revenue) and over-stocking of perishables (waste), the two things that hurt q-commerce margins most;
- smooth the morning rush by knowing the recurring load in advance and staffing/routing against it;
- and spot demand patterns at the group level — *this neighbourhood of shared flats reliably reorders rice every other Sunday* — that simply aren't visible when every order looks like an anonymous individual.

No competitor without a group-ordering layer generates this signal, because individual orders don't announce themselves a day ahead. Squad does. That's a forecasting edge that compounds quietly the more squads exist, and it's the kind of structural data advantage that's very hard to copy after the fact.

---

16. Making money

The instinct to monetize members would be a mistake — the moment you cap how many people can be in a squad, you've kneecapped the exact viral loop that makes the feature valuable. Membership stays free, always. The money comes from depth, not access.

Lever	How it's priced	Why
Members	Always free	A paywall here kills the network effect
Advanced analytics	Premium	Spend breakdowns and trends for the people who want them
Ledger export	3×/month free, then paid	Doesn't touch the core loop; gentle nudge to upgrade
Squad-level Swiggy One	Subscription	Free delivery + perks shared across the flat (see retention)
Priority delivery slots	Premium	Squad-level priority during the rush
Office billing	Enterprise	Office squads are a natural B2B on-ramp

The pricing detail that matters most: premium should be **per-squad, not per-person.** One motivated flatmate pays and the whole squad gets the perks. For a target audience of students and bachelors counting every rupee, "one of us covers it and we all benefit" is a far easier yes than asking five people to each subscribe.

---

17. Why people stay

The growth loop is unusually clean because the invite *is* the product. When Priya adds Raj to Home Squad, she's not handing him a "₹100 off" code he'll ignore — she's pulling him into something he already does with her every week. The intent is real and the context is concrete, which is why this kind of invite converts so much better than a discount referral, and why every new squad quietly drags its members further into Swiggy.

Keeping them is about making the squad feel like a *thing that's alive* rather than a calculator they open occasionally. Subscriptions create a standing reason to order. The ledger creates a standing reason to come back and settle. But the strongest hooks come from making the squad's *value* visible and from giving it small, timely reasons to act:

Show the squad what it's saving. This is the one I'd build first. Every time a squad orders together, it saves something real — a delivery fee split four ways, a bulk price, an offer that applied to the whole cart. Right now that saving is invisible, so nobody feels it. Squad should surface it plainly and let it accumulate: *"Ordering as a squad saved you ₹430 this month"* and *"₹2,900 saved since you started."* Where it's fair to, it can even contrast against going it alone or against typical market rates — *"about ₹120 cheaper than three separate orders."* A running savings counter turns the squad from a convenience into something people feel financially attached to, and attachment is what survives a competitor's launch.

Turn price drops into a nudge. Swiggy already knows its own prices. If potatoes were ₹20 yesterday and they're ₹15 today, that's a reason to act *now*, and a squad is exactly the place to surface it — *"potatoes dropped ₹5 today, want to add them to the run?"* It's a demand-generation lever, not just retention: it gives an idle squad a concrete reason to open a cart, and it trains people to treat Squad as the place where the good calls happen.

Re-engage a quiet squad deliberately. When a squad goes silent — nobody's ordered in a couple of weeks — that's a signal, and it's worth acting on rather than letting the squad drift. A well-timed nudge, and where the economics justify it a small voucher, can pull it back to life. The one caution I'd put on this in writing: it has to be tied to the squad-health signal and used sparingly, because a predictable "wait two weeks, get a voucher" pattern just trains people to wait. Used as genuine re-engagement, not a standing giveaway, it works.

Make Swiggy One a shared benefit. A Swiggy One membership held at the squad level — free delivery and perks shared across the flat — is both a monetization line and a retention hook. It gives the squad a collective reason to exist beyond splitting bills, and "we all get free delivery through the squad" is a sticky little fact that keeps people in.

One honest note on all of the above: savings counters, price nudges, and vouchers are powerful but they touch margin, so they should be modelled against unit economics rather than treated as free wins. The savings *visibility* costs almost nothing and I'd lean on it hardest; the discounting levers want a careful hand.

A few things worth measuring to know if any of this is working: how many squads get created and how many survive; how many people end up in each; whether squad members order more often and spend more than they did solo; how many subscriptions a squad runs; whether balances actually get settled; the cumulative savings a squad has racked up (a strong leading indicator of attachment); and how cleanly an invite turns into an installed, ordering member. The retention curve at 30, 60, 90 days is the one that tells the real story.

---

18. What could go wrong

I'd rather name the weak points than pretend they aren't there — a feature like this has a few real ones.

The scariest is the **cold start inside a squad.** The whole thing assumes your flatmates are on Swiggy. If three of five are on Blinkit, the shared cart is half-empty and the value collapses. This is genuinely the make-or-break risk, and the answer has to be that the invite link is treated as a first-class acquisition channel, not an afterthought — and that a squad still does something useful even at partial adoption, so it's not all-or-nothing.

Close behind is **the squad dying when its most active person leaves.** Groups tend to have one engine — the person who always starts the order. The Pilot model spreads that load better than a single-admin design would, mandatory succession keeps the squad from collapsing on an exit, and the health nudges help, but I won't pretend this is fully solved. It's the thing I'd watch hardest after launch.

The rest are more manageable. **Competitors will copy this** within a year or so — but the moat isn't the feature, it's owning the transaction, and an expense app can't follow Swiggy onto that ground. **Complexity bleeding into the solo flow** is a real danger, handled by keeping Squad strictly opt-in and invisible to anyone without an active squad. **Payments regulation** is sidestepped for now by refusing to pool money. **Disputes and non-payment** are softened by transparency and the settle-before-leave gate rather than coercion. And the **real-time engineering** is non-trivial but it's a proven pattern, not a research project — the risk is in scoping it tightly, not in whether it can be done.

19. The honest case for building it

Stripping away the optimism, here's my honest read on whether this clears the bar.

On the fundamentals, it's strong. The **problem is real and frequent** — not a vitamin, a painkiller for anyone who's shared a flat. The **user is already Swiggy's user**, so there's no new audience to go chase; this deepens a cohort Swiggy already has and wants to keep. And the **platform fit is the rare kind that's structural** — this feature can only really exist where the order and the payment are the same event, which is Swiggy's turf and almost nobody else's. That combination is what makes me think it's worth doing rather than just interesting.

The **timing is good but not permanent.** UPI and co-living density have opened the window now; the same logic means a competitor can copy the surface feature within a year or so. The defense isn't the feature, it's the transaction ownership underneath it — but that's a reason to move, not to wait.

I won't oversell the soft spots. The feature **leans on motivated people** — the flatmate who keeps the squad alive — and if they leave, squads can wither. The Pilot model and succession logic blunt this, but it's the genuine open question, and it's a *retention* risk more than a *concept* risk. And the **real-time cart is real work**, though it's well-trodden engineering rather than anything speculative.

Put it together and my view is straightforward: the upside is strategic and defensible, the risks are real but bounded and mostly addressable in design, and the cost to find out is a tightly-scoped MVP rather than a moonshot. **It's worth building** — and the cleanest way to prove it is to ship the core loop to a few hundred real shared flats and watch whether the squads survive past month one.

> *The whole thing in one line: every other tool splits the bill after you've bought. Squad does it while you buy — and only the platform that owns the checkout can.*

20. Roadmap

Rough sequencing, assuming Builders Club access to the Food and Instamart MCP servers. Timelines are indicative — the point is the order, not the exact months.

Phase	When	What lands
Core loop	Months 0–3	Squad creation, live shared cart, Pilot split, equal/custom/item-level, basic ledger, manual UPI/cash settlement, squad chat
Stickiness	Months 3–6	Recurring subscriptions, squad savings counter, price-drop nudges, reminders, monthly reports + export caps, admin succession
Intelligence	Months 6–12	Split suggestions, market-rate comparisons, idle-squad re-engagement, Instamart demand-forecasting feed, office billing

References

Market figures cited in this document are drawn from public 2025–26 research and industry reporting, including:

- India online food delivery market size — IMARC, Expert Market Research (2025)
- Swiggy / Zomato GOV share — Datum Intelligence (Oct 2025)
- India quick commerce GMV & Instamart dark-store / share figures — GlobeNewswire / ResearchAndMarkets, Mordor Intelligence, Demandsage, Reuters via webbytemplate (2025–26)
- India co-living bed demand & migrant/student population — Colliers India, Teji Mandi / JLL (2025)
- UPI transaction volume, value, and bill-split trend — NPCI via Meetanshi, Elets BFSI, IBEF, Coinlaw (2025–26)

Figures are indicative of market scale and direction; exact values vary by source and methodology.
